AUTOMATED BOOKKEEPING SYSTEM IN PYTHON

1. PROJECT OVERVIEW

What This System Does

This project is a complete automated bookkeeping pipeline written in Python. Its purpose is to replace the manual process of recording, categorising, and reporting on financial transactions for a small business or freelancer. Instead of opening spreadsheets and manually entering every bank charge, client payment, and software subscription, this system does it all automatically — from raw import to a finished Excel report package.

The system handles five distinct stages, each built from one or more functions:

Stage	Functions Involved	Output
1. Import	import_bank_transactions, extract_from_pdf	Standardised DataFrame
2. Categorise	create_categorization_rules, categorize_transactions, learn_from_manual_categories	Labelled transactions
3. Ledger	create_chart_of_accounts, map_category_to_account, create_journal_entries	Double-entry journal
4. Reporting	generate_income_statement, generate_balance_sheet	Financial statements
5. Export	export_bookkeeping_package	Multi-sheet Excel file

Core Technology Choices

Library	Why It Is Used
pandas	The backbone of the whole system. Every transaction list, journal, and financial statement is a pandas DataFrame. It makes filtering, grouping, and aggregating financial data simple.
numpy	Provides efficient numerical operations used implicitly by pandas for summing and calculating financial metrics.
openpyxl	Allows pandas to write multi-sheet Excel workbooks. Without it, the export step cannot produce .xlsx files.
python-dateutil	Parses date strings from bank statements that come in inconsistent formats — used internally by pd.to_datetime.
pdfplumber	Opens PDF bank statements and extracts their raw text page by page, which the import function then parses into rows.

The Sign Convention Used Throughout

> ⚠ This is the most important assumption in the entire system. Every function depends on it being consistent.

Negative amounts represent money leaving the account (expenses, fees, purchases). Positive amounts represent money entering the account (client payments, refunds, deposits). This convention is established during import and never changed. If a source file uses the opposite convention, every downstream function will produce wrong results silently.

2. DATA IMPORT FUNCTIONS

2.1 import_bank_transactions (file_path, file_type)

Purpose

This is the entry point of the entire pipeline. It reads a bank statement or credit card export in CSV, Excel, or PDF format and returns a clean, standardised DataFrame that every later function can work with reliably.

Function Signature

```
def import_bank_transactions(file_path, file_type='csv'):
```

Parameters

Parameter	Type	Default	Description
file_path	str	—	Path to the file on disk, e.g. 'bank_statement.csv'
file_type	str	'csv'	One of 'csv', 'excel', or 'pdf'. Controls which reader is called.

Step-by-Step Logic

Step 1 — Choose the right reader

```
if file_type == 'csv':
    df = pd.read_csv(file_path)
elif file_type == 'excel':
    df = pd.read_excel(file_path)
elif file_type == 'pdf':
    df = extract_from_pdf(file_path)
```

A simple branch selects the correct pandas reader. For PDF files, it delegates to the separate extract_from_pdf function. There is no handling for unrecognised file types — it will raise an error or return an empty result silently.

Step 2 — Standardise column names

```
column_mapping = {
    'Date': 'date',
    'Transaction Date': 'date',
    'Description': 'description',
    'Memo': 'description',
    'Amount': 'amount',
    'Debit': 'debit',
    'Credit': 'credit'
}
```

```
df.rename(columns=column_mapping, inplace=True)
```

Different banks export different column names. Barclays may call it 'Transaction Date'; HSBC might say 'Date'. This mapping normalises all known variants to three internal names: date, description, amount. Any column not in the mapping keeps its original name — this can cause silent failures if a required column is named something unexpected.

Step 3 — Convert dates

```
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

Converts the date column from raw strings (e.g. '15/03/2025', 'Mar 15 2025') to proper datetime objects. The errors='coerce' argument means any date that cannot be parsed becomes NaT (Not a Time — a null date). This is important: it means malformed dates fail silently rather than crashing the import. These rows with null dates will appear in all downstream data, potentially skewing date-filtered reports.

Step 4 — Merge debit and credit columns

```
if 'debit' in df.columns and 'credit' in df.columns:
    df['amount'] = df['credit'].fillna(0) - df['debit'].fillna(0)
    df.drop(['debit', 'credit'], axis=1, inplace=True)
```

Some banks export separate Debit and Credit columns instead of a single signed Amount column. This step merges them using the formula: amount = credit − debit. The fillna(0) prevents NaN propagation when one column is blank for a given row. After merging, the original debit and credit columns are removed so the rest of the pipeline sees only one amount column. The result preserves the sign convention: credits are positive, debits are negative.

Step 5 — Normalise descriptions

```
df['description'] = df['description'].str.strip().str.upper()
```

Strips leading and trailing whitespace, then converts everything to uppercase. This is critical for the keyword-matching categorisation step — without it, 'Starbucks', 'STARBUCKS', and ' starbucks ' would all be treated as different strings and only one would match the rule. This one line prevents the majority of categorisation misses.

Step 6 — Sort and return

```
return df.sort_values('date').reset_index(drop=True)
```

Sorts all rows chronologically and resets the row index to 0, 1, 2, … so later code can safely use positional references. The original file order is discarded.

Assumptions

- The file exists at the path given and is readable.
- At minimum, the file contains columns that map to date, description, and either amount or both debit and credit.
- All date formats can be auto-detected by pandas — exotic formats may silently produce null dates.
- Expenses are negative and income is positive after the debit/credit merge.
- Credit card exports should be passed in separately and later combined with pd.concat.

2.2 extract_from_pdf (pdf_path)

Purpose

Acts as a specialised reader for PDF bank statements. It is called automatically by import_bank_transactions when file_type is 'pdf'. It opens the PDF, reads every page, and attempts to extract rows of transaction data from the raw text.

Step-by-Step Logic

Step 1 — Open the PDF and loop through pages

```
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        text = page.extract_text()
```

pdfplumber opens the file and exposes each page as an object. extract_text() returns the entire page as a single string, preserving newlines. The 'with' block ensures the file handle is properly closed even if an error occurs mid-read.

Step 2 — Split into lines and parse each one

```
lines = text.split('\n')
for line in lines:
    parts = line.split()
    if len(parts) >= 3:
        transactions.append({
            'date': parts[0],
            'description': ' '.join(parts[1:-1]),
            'amount': parts[-1]
        })
```

Each line is split on whitespace. The rule is: first token is the date, last token is the amount, and everything in between is the description. The check len(parts) >= 3 filters out blank lines, headers, footers, and single-word lines. This is a template — real PDF statements vary enormously and will need custom parsing logic here.

Critical Limitation

> ⚠ This function is a stub. It will produce garbage output on most real bank PDFs without customisation for your specific bank's layout. Scanned PDFs (images) will return empty text since pdfplumber cannot perform OCR.

3. CATEGORISATION FUNCTIONS

3.1 create_categorization_rules ( )

Purpose

Returns a plain Python dictionary that acts as the rule book for automatic categorisation. Each key is a human-readable category name and each value is a list of keyword strings to search for inside transaction descriptions.

The Rules Dictionary — What It Contains

Category	Sample Keywords
Office Supplies	STAPLES, OFFICE DEPOT, AMAZON
Utilities	ELECTRIC, GAS COMPANY, WATER, INTERNET
Marketing	GOOGLE ADS, FACEBOOK, LINKEDIN, MAILCHIMP
Software	MICROSOFT, ADOBE, DROPBOX, SLACK
Meals & Entertainment	RESTAURANT, COFFEE, STARBUCKS, LUNCH
Travel	AIRLINE, HOTEL, UBER, LYFT, RENTAL CAR
Professional Services	LEGAL, ACCOUNTING, CONSULTING
Rent	LANDLORD, PROPERTY MANAGEMENT, RENT
Insurance	INSURANCE
Bank Fees	MONTHLY FEE, SERVICE CHARGE, ATM FEE
Client Payment	PAYMENT RECEIVED, INVOICE, DEPOSIT
Owner Draw	TRANSFER TO PERSONAL, OWNER WITHDRAWAL

Why It Is a Separate Function

Isolating the rules into their own function makes them easy to edit without touching any logic code. You can add a new category, expand a keyword list, or remove a rule in one place and it immediately affects the entire pipeline. It also makes the rules easy to load from a config file or database in future.

Key Assumption

- All keywords are uppercase, which matches the normalised descriptions produced by import_bank_transactions. If you add lowercase keywords here they will never match.

3.2 categorize_transactions (df, rules)

Purpose

Takes the standardised DataFrame of transactions and the rules dictionary, then assigns a category and a confidence score to every row using keyword matching and amount-based fallbacks.

Step-by-Step Logic

Step 1 — Initialise all rows as Uncategorized

```
df['category'] = 'Uncategorized'
df['confidence'] = 0.0
```

Before any matching begins, every row is set to 'Uncategorized' with a confidence of 0.0. This is the baseline state. Subsequent steps will overwrite only the rows they can match.

Step 2 — Keyword matching loop

```
for category, keywords in rules.items():
    for keyword in keywords:
        mask = df['description'].str.contains(keyword, case=False, na=False)
        uncategorized = df['category'] == 'Uncategorized'
        df.loc[mask & uncategorized, 'category'] = category
        df.loc[mask & uncategorized, 'confidence'] = 0.9
```

For every category and every keyword within that category, a boolean mask is built using str.contains. The case=False argument makes matching case-insensitive even though descriptions are already uppercase — this is a safety net. The na=False argument ensures NaN descriptions do not raise errors. The key design decision is the uncategorized guard: a row can only be categorised once, and the first matching keyword wins. This means the order of categories in the rules dictionary matters. If AMAZON appears under Office Supplies and also could be Travel, whichever category is listed first in the dictionary claims all AMAZON transactions.

Step 3 — Fallback by amount sign

```
df.loc[(df['amount'] > 0) & (df['category'] == 'Uncategorized'), 'category'] =
'Income'
df.loc[(df['amount'] < 0) & (df['category'] == 'Uncategorized'), 'category'] = 'Other
Expense'
```

After keyword matching, any rows still Uncategorized are given a rough fallback category based purely on the sign of the amount. Positive remaining = Income. Negative remaining = Other Expense. These fallbacks deliberately have no confidence score update — they stay at 0.0, which means they will appear on the Review Required sheet in the final export.

Confidence Score Meaning

Score	Source	Meaning	Export Sheet
0.0	Fallback (amount sign)	Very rough guess — needs human review	Review Required

0.9	Keyword match	High confidence but not verified	Review Required
1.0	Manual correction history	Human-confirmed — final	Not flagged

3.3 learn_from_manual_categories (df, manual_categories_file)

Purpose

Applies previously confirmed human categorisations to new transactions. When a bookkeeper corrects a category and saves the result, this function reads those corrections and re-applies them automatically next time the same transaction description appears.

Step-by-Step Logic

Step 1 — Load the corrections file

```
try:
    manual = pd.read_csv(manual_categories_file)
except FileNotFoundError:
    print('No manual categorization history found')
```

Attempts to read a CSV file containing past human corrections. The try/except means the whole pipeline still runs even on the very first use, before any corrections have been saved. The FileNotFoundError is caught gracefully.

Step 2 — Build a description-to-category mapping

```
category_map = {}
for _, row in manual.iterrows():
    category_map[row['description']] = row['category']
```

Converts the CSV into a plain dictionary. If the same description appears multiple times in the corrections file (a later correction overrides an earlier one), the last entry wins due to Python dictionary key overwriting.

Step 3 — Apply the learned categories

```
for desc, cat in category_map.items():
    mask = df['description'] == desc
    df.loc[mask, 'category'] = cat
    df.loc[mask, 'confidence'] = 1.0
```

For every learned mapping, finds all rows with an exactly matching description and overwrites both the category and the confidence score. The confidence becomes 1.0, meaning these rows will not appear on the Review Required sheet.

Critical Limitation

> ⚠ Matching is done on exact string equality. A description of 'GOOGLE ADS - ACCOUNT 12345' will match a saved correction only if the correction file contains that exact string. Any variation — a different account number, extra spaces, different wording — will fail to match silently.

4. LEDGER FUNCTIONS

4.1 create_chart_of_accounts ( )

Purpose

Returns a dictionary that defines the accounting structure of the business. Every account has a unique numeric code, a human-readable name, and a type classification. This is the reference that all journal entry and reporting functions use to know where money belongs.

Account Code Numbering Convention

The codes follow standard accounting practice where the leading digit defines the account type:

Code Range	Account Type	Normal Balance	Example
1xxx	Assets	Debit (positive)	Cash, Receivables
2xxx	Liabilities	Credit (negative)	Accounts Payable, Credit Card
3xxx	Equity	Credit (negative)	Owner Equity, Retained Earnings
4xxx	Income / Revenue	Credit (negative)	Service Revenue, Product Sales
5xxx	Expenses	Debit (positive)	Rent, Software, Marketing

This leading-digit convention is critical because the generate_income_statement and generate_balance_sheet functions use str.startswith('4') and str.startswith('5') to filter accounts. If you use integer codes instead of string codes, or change the numbering scheme, those filters will break silently and produce empty or wrong financial statements.

Full Account List

Code	Name	Type	Notes
1000	Cash - Checking	Asset	All income and expense transactions flow through this account
1010	Cash - Savings	Asset	Defined but never used by the journal entry logic
1200	Accounts Receivable	Asset	Defined but never automatically populated
2000	Accounts Payable	Liability	Defined but never used — system is cash-basis only
2100	Credit Card Payable	Liability	Defined but never automatically populated
3000	Owner Equity	Equity	Starting equity — not adjusted by the system

3100	Retained Earnings	Equity	Net income is never automatically closed here
4000	Service Revenue	Income	Default income account for all client payments
4100	Product Sales	Income	Defined but not mapped in category_to_account
5000–5900	Expense Accounts	Expense	One account per expense category

4.2 map_category_to_account (category, chart_of_accounts)

Purpose

Translates a human-readable category name such as 'Marketing' into its corresponding account code such as '5200'. This is the bridge between the categorisation layer and the accounting ledger.

How It Works

```
category_mapping = {
    'Office Supplies': '5000',
    'Utilities':       '5100',
    'Marketing':       '5200',
    'Software':        '5300',
    'Meals & Entertainment': '5400',
    'Travel':          '5500',
    'Professional Services': '5600',
    'Rent':            '5700',
    'Insurance':       '5800',
    'Bank Fees':       '5900',
    'Client Payment':  '4000',
    'Income':          '4000',
    'Other Expense':   '5000'
}
return category_mapping.get(category, '5000')
```

The function is a simple dictionary lookup. The default fallback is account '5000' (Office Supplies). This means any category that is not in the mapping — including new ones you add to the rules without updating this function — will silently post to Office Supplies, distorting your expense breakdown.

Silent Failure Risk

> ⚠ If you add a new category in create_categorization_rules but forget to add it here, every transaction in that category will be quietly filed under Office Supplies (5000) with no warning. Always update both functions together.

4.3 create_journal_entries (transactions_df, chart_of_accounts)

Purpose

This is the accounting engine of the system. It converts every transaction in the DataFrame into a proper double-entry journal entry — two rows per transaction that together keep the books in balance.

Double-Entry Bookkeeping — The Concept

In double-entry bookkeeping, every transaction affects two accounts simultaneously. For every debit there is an equal credit. This ensures the fundamental accounting equation always holds: Assets = Liabilities + Equity. This is not just a formality — it makes errors detectable. If the books do not balance, something is wrong.

Step-by-Step Logic

Step 1 — Initialise the loop

```
journal_entries = []
entry_number = 1
```

An empty list collects all journal entry rows as dictionaries. The entry_number counter ties together the two rows of each transaction. Both rows of the same transaction share the same entry number, allowing you to find the matching debit and credit for any entry.

Step 2 — For each transaction, get the account code and clean the amount

```
for _, transaction in transactions_df.iterrows():
    account_code = map_category_to_account(transaction['category'], chart_of_accounts)
    amount = abs(transaction['amount'])
```

The map_category_to_account call converts the category label into the correct account code. abs() removes the sign from the amount because the sign is now carried by which column the amount appears in — debit or credit — not by positive/negative.

Step 3 — Expense transaction (amount was negative)

```
if transaction['amount'] < 0:
    # Row 1: Debit the expense account
    journal_entries.append({
        'account_code': account_code,   # e.g. 5300 Software
        'debit': amount,
        'credit': 0,
        ... })
    # Row 2: Credit cash
    journal_entries.append({
        'account_code': '1000',         # Cash - Checking
        'debit': 0,
        'credit': amount,
        ... })
```

When money leaves the account (an expense), the expense account is debited (it increases — expenses grow with debits) and cash is credited (it decreases — assets shrink with credits). This is the standard accounting treatment for a cash expense.

Step 4 — Income transaction (amount was positive)

```
else:
    # Row 1: Debit cash
    journal_entries.append({
        'account_code': '1000',         # Cash - Checking
        'debit': amount,
        'credit': 0,
        ... })
    # Row 2: Credit the revenue account
    journal_entries.append({
        'account_code': account_code,   # e.g. 4000 Service Revenue
        'debit': 0,
        'credit': amount,
        ... })
```

When money enters the account (income), cash is debited (assets increase with debits) and revenue is credited (income grows with credits). This is the standard accounting treatment for a cash receipt.

Visual Example — Two Transactions

Entry #	Account	Debit ($)	Credit ($)
1	Software Expense (5300) — expense row	200.00	—
1	Cash - Checking (1000) — expense row	—	200.00
2	Cash - Checking (1000) — income row	1,500.00	—
2	Service Revenue (4000) — income row	—	1,500.00

Assumptions and Limitations

- The system is pure cash-basis accounting. Expenses are recorded when cash leaves, income when cash arrives. There is no accrual, no accounts payable, no accounts receivable workflow.
- All money is assumed to move through a single bank account: 1000 Cash - Checking. Multi-account businesses need modifications.
- Credit card transactions are treated as immediate cash outflows even though technically they create a liability. The credit card payable account (2100) is never used.
- Transactions with amount = 0 fall into the else (income) branch and create a zero-value journal entry. This is harmless but creates noise.

5. FINANCIAL STATEMENT FUNCTIONS

5.1 generate_income_statement (journal_df, start_date, end_date)

Purpose

Produces a formatted income statement (also called a Profit and Loss report) for a specific time period. It answers the question: did the business make money or lose money between these two dates, and which specific categories drove the result?

Step-by-Step Logic

Step 1 — Filter to the reporting period

```
period_journal = journal_df[
    (journal_df['date'] >= start_date) &
    (journal_df['date'] <= end_date)
]
```

Creates a filtered view of the journal containing only rows within the date range (inclusive on both ends). This is a pandas boolean filter — it does not modify the original journal. Rows with NaT dates will fail both conditions and be excluded, which is the correct behaviour.

Step 2 — Sum income accounts

```
income_accounts = period_journal[
    period_journal['account_code'].str.startswith('4')
]
total_income = income_accounts['credit'].sum() - income_accounts['debit'].sum()
```

Filters to only rows from accounts starting with '4' (revenue accounts). Total income is calculated as credits minus debits on those accounts. This is the standard method: revenue normally has a credit balance, but any refunds issued would be debits to the revenue account. Subtracting debits nets those out automatically.

Step 3 — Sum expenses by account

```
expense_accounts = period_journal[
    period_journal['account_code'].str.startswith('5')
]
expenses_by_account = expense_accounts.groupby('account_name').agg({
    'debit': 'sum',
    'credit': 'sum'
})
expenses_by_account['net'] = expenses_by_account['debit'] -
expenses_by_account['credit']
total_expenses = expenses_by_account['net'].sum()
```

Filters to only rows from accounts starting with '5' (expense accounts). Rather than a single total, expenses are grouped by account name so the report shows a line for each expense type. Net expense per account is debits minus credits — expense refunds would reduce the net correctly. The sum of all net expense lines gives total expenses.

Step 4 — Calculate net income

```
net_income = total_income - total_expenses
```

The classic formula. A positive result means profit; a negative result means a loss for the period.

Step 5 — Format for display

The function builds a two-column DataFrame with account names on the left and formatted dollar strings on the right. This is purely for display — the Amount column contains strings like '$5,000.00', not numbers. You cannot perform calculations on this output.

Important Limitations

- This is a cash-basis income statement only. Invoices sent but not yet paid do not appear as income. Bills received but not yet paid do not appear as expenses.
- The formatted Amount column is a string. To do further calculation you must use the numeric variables returned by the function, not the DataFrame.
- No tax calculation, depreciation, or amortisation is performed.

5.2 generate_balance_sheet (journal_df, as_of_date)

Purpose

Produces a balance sheet snapshot as of a specific date. Unlike the income statement which covers a period, the balance sheet is a point-in-time view. It answers: what does the business own (assets), what does it owe (liabilities), and what is the owner's stake (equity) at this exact moment?

Step-by-Step Logic

Step 1 — Include all history up to the snapshot date

```
period_journal = journal_df[journal_df['date'] <= as_of_date]
```

This is the most important difference from the income statement filter. The balance sheet includes everything from the very first transaction ever recorded up to the snapshot date. Asset and liability balances are cumulative — they do not reset at the start of a new year.

Step 2 — Calculate balances for every account

```
balances = period_journal.groupby(['account_code', 'account_name']).agg({
    'debit': 'sum',
    'credit': 'sum'
})
balances['balance'] = balances['debit'] - balances['credit']
```

Groups the entire journal history by account, summing all debits and all credits. Balance = total debits minus total credits. For asset accounts (like Cash), this gives a positive number representing what is

held. For liability and equity accounts, the result is negative by convention — which is why abs() is applied in the next step.

Step 3 — Split by account type

```
assets = balances[balances.index.get_level_values(0).str.startswith('1')]
liabilities = balances[balances.index.get_level_values(0).str.startswith('2')]
equity = balances[balances.index.get_level_values(0).str.startswith('3')]
```

Uses the account code prefix to split the full balance table into three groups. The index.get_level_values(0) call accesses the first level of the multi-level index created by groupby — this is the account code level.

Step 4 — Total each section

```
total_assets = assets['balance'].sum()
total_liabilities = abs(liabilities['balance'].sum())
total_equity = abs(equity['balance'].sum())
```

abs() is applied to liabilities and equity because their natural accounting balance is a credit (negative in the debit−credit calculation). Without abs(), these would display as negative numbers, which is confusing in a report.

The Balance Sheet Equation

> ℹ Assets = Liabilities + Equity. This system calculates all three independently but does NOT verify they balance. If they do not balance, there is an error in the data — typically because net income has not been posted to retained earnings.

Critical Limitation — No Closing Entry

In standard accounting, at the end of each period a closing entry is posted that moves net income from the income accounts into retained earnings (account 3100 in this system). This system never does that. As a result, the equity section of the balance sheet will not include the current period's profit or loss. The balance sheet will not balance without this closing entry.

6. EXPORT FUNCTION

6.1 export_bookkeeping_package (transactions, journal, income_stmt, balance_sheet, filename)

Purpose

Bundles all produced data into a single professionally organised Excel workbook. This is the final deliverable of the pipeline — the file handed to an accountant, used for tax preparation, or archived as the period's financial record.

Step-by-Step Logic

Step 1 — Open the Excel writer

```
with pd.ExcelWriter(filename, engine='openpyxl') as writer:
```

Opens an ExcelWriter context that manages a single Excel workbook for writing. The engine='openpyxl' argument specifies the library used to build the file. The 'with' block ensures the workbook is saved and closed properly even if an error occurs while writing. If the filename already exists, it is overwritten without warning.

Step 2 — Write the Transactions sheet

```
transactions.to_excel(writer, sheet_name='Transactions', index=False)
```

Writes the complete transactions DataFrame including date, description, amount, category, and confidence score. index=False prevents pandas from writing the row numbers as an extra column. This sheet shows every transaction imported for the period and is the raw data layer of the report.

Step 3 — Write the Journal Entries sheet

```
journal.to_excel(writer, sheet_name='Journal Entries', index=False)
```

Writes the full double-entry journal with all debit and credit rows. This is the accounting ledger — useful for accountants to verify the books, trace individual entries, or import into accounting software. Each pair of rows sharing an entry_number represents one original transaction.

Step 4 — Write the Income Statement sheet

```
income_stmt.to_excel(writer, sheet_name='Income Statement', index=False)
```

Writes the formatted income statement DataFrame. Because the Amount column contains strings with dollar signs, this sheet is for reading only — it cannot be used for further calculation in Excel without reformatting.

Step 5 — Write the Balance Sheet sheet

```
bs_summary = pd.DataFrame({
```

```
    'Account Type': ['Assets', 'Liabilities', 'Equity'],
    'Balance': [balance_sheet['Total Assets'],
                balance_sheet['Total Liabilities'],
                balance_sheet['Total Equity']]
})
bs_summary.to_excel(writer, sheet_name='Balance Sheet', index=False)
```

The balance sheet dictionary returned by generate_balance_sheet is converted into a simple three-row summary DataFrame before writing. Only the totals are written — the detailed account-by-account breakdown stored in the dictionary's Assets, Liabilities, and Equity keys is not included in this export.

Step 6 — Write the Review Required sheet

```
needs_review = transactions[transactions['confidence'] < 0.9]
needs_review.to_excel(writer, sheet_name='Review Required', index=False)
```

Filters the transactions DataFrame to only rows with confidence below 0.9 — meaning any transaction that was not manually confirmed previously. This creates a focused action list for the bookkeeper. Only transactions on this sheet need human attention; everything else has already been verified.

Output File Structure

Sheet Name	Contents & Purpose
Transactions	Every transaction with category and confidence. The complete record of all imports.
Journal Entries	Double-entry ledger rows, two per transaction. Used by accountants and for audit trail.
Income Statement	Revenue minus expenses equals net income for the period. Human-readable formatted report.
Balance Sheet	Total assets, liabilities, and equity as of the period end date.
Review Required	Only the transactions needing human correction. The action list for the bookkeeper.

Limitations

- No visual formatting is applied — column widths, bold headers, number formats, and colours are all default. The output is functional but not polished.
- Overwriting an existing file happens silently with no confirmation prompt.
- The filename parameter defaults to 'bookkeeping_package.xlsx' if not provided.

7. FULL SYSTEM DATA FLOW

How All Functions Connect

The pipeline is strictly linear. The output of each stage becomes the input of the next. No function reads from disk after the initial import — all data passes in memory through the function chain.

Stage & Function	Input → Output
import_bank_transactions (×N sources)	Raw CSV / Excel / PDF files → Standardised DataFrame per source
pd.concat (built-in)	Multiple DataFrames → Single combined transactions DataFrame
create_categorization_rules	Nothing → Rules dictionary
categorize_transactions	Transactions DataFrame + Rules → Transactions with category & confidence
learn_from_manual_categories	Categorised transactions + manual CSV → Transactions with 1.0 confidence overrides
create_chart_of_accounts	Nothing → Account code dictionary
create_journal_entries	Categorised transactions + Chart of accounts → Journal DataFrame (2 rows per transaction)
generate_income_statement	Journal DataFrame + date range → Formatted income statement DataFrame + net income float
generate_balance_sheet	Journal DataFrame + snapshot date → Balance sheet dictionary
export_bookkeeping_package	All of the above → bookkeeping_package.xlsx

The Central Role of the Journal

The journal DataFrame produced by create_journal_entries is the single most important artifact in the system. Both financial statement functions read exclusively from the journal, not from the original transactions. This means any error that enters the journal — a wrong account code, a miscategorised transaction, a missing entry — will propagate directly into both the income statement and the balance sheet.

8. KEY ASSUMPTIONS & LIMITATIONS

Global Assumptions

Assumption	Where It Applies	What Breaks If Wrong
Negative = expense, positive = income	Every function	Journal entries are backwards; statements are inverted
Account codes are strings starting with 1/2/3/4/5	generate_income_statement, generate_balance_sheet	str.startswith filters return empty DataFrames
Single cash account (1000)	create_journal_entries	All transactions post to the same account regardless of source
Cash-basis accounting only	Entire pipeline	Unpaid invoices and outstanding bills are invisible
Descriptions are uppercase	categorize_transactions, learn_from_manual_categories	Keyword matching and exact-match learning both fail
No closing entry for net income	generate_balance_sheet	Balance sheet equity excludes current period profit/loss

Best Practices for Safe Use

1. Always reconcile the exported journal against your actual bank statement before treating any report as final.
2. Review the Review Required sheet after every run. These are the transactions the system is least confident about.
3. When adding a new spending category, update create_categorization_rules and map_category_to_account together in the same edit session.
4. Store the manual_categories.csv file under version control so corrections are never lost.
5. Never use the formatted income statement Amount column for further calculation — use the numeric return values instead.
6. For any business with credit cards, loans, or outstanding invoices, this cash-basis system is not sufficient for statutory accounts without modification.